PART A: BUG FIXES

BUG-001 — Incorrect storage usage and available-space calculations

Feature/location: Storage panel

PDF Visual Reference: BUG-001, Page 1

Current behaviour:

The initial display shows approximately 250 MB used and 38.4 GB available, even when there are no saved projects.

After saving a project, the used figure changes to an apparently unrelated value, such as 81.7 KB.

The available figure remains at 38.4 GB instead of decreasing to reflect the newly saved project.

Expected behaviour:

Storage used must reflect the actual space occupied by saved resources.

Storage available must decrease as resources are saved.

Both figures must remain mathematically consistent with the defined total capacity.

Required correction:

Investigate the existing storage calculation and persistence mechanisms. Correct the usage measurement and available-space calculation. Do not use arbitrary, hardcoded or unrelated values.

Acceptance criteria:

Saving a project increases storage used by the actual amount retained.

Available storage decreases by the corresponding amount.

Reopening an existing project does not count it again as a new project.

Deleting a saved resource releases its occupied space.

Figures remain accurate after refreshing or reopening the app.

Existing feature enhancements

ENHANCEMENT-001 — Complete storage monitoring and management

Feature/location: Storage panel and saved-resource management

PDF Visual Reference: ENHANCEMENT-001, Page 1

Required enhancement:

Establish a clearly defined storage capacity for Batch Studio.

Track all resources retained by the application, including projects, folders, documents, images, templates and uploaded fonts.

Display the actual storage size occupied by each saved item.

Display the total storage used.

Calculate and display the remaining available capacity.

Provide a way to identify resources consuming storage.

Allow users to delete unwanted saved projects and resources to reclaim space.

Update storage figures whenever resources are added, modified or deleted.

Important implementation requirement: The storage calculation must account for the actual persistence system Batch Studio uses. The AI must inspect the architecture before deciding how to measure capacity and usage. It must not assume that a browser quota estimate is automatically the app's own storage capacity.

BUG-002 — Remove unnecessary bottom page-indicator dots

Feature/location: Bottom of the affected page

PDF Visual Reference: BUG-002, Page 2

Current behaviour:

Four unnecessary dots appear at the bottom of the interface, resembling a mobile home-screen page indicator. They create the impression that the interface contains multiple horizontally navigable pages.

Expected behaviour:

The unnecessary dots should not appear.

Required correction:

Remove the four-dot indicator and any associated animation or movement, provided it is not required by another working feature.

Acceptance criteria:

The four dots are no longer visible.

No unnecessary horizontal page-indicator animation remains.

Existing navigation and scrolling functionality remain unaffected.

BUG-003 — Project action menu is partially hidden or clipped

Feature/location: Project card, three-dot action menu

PDF Visual Reference: BUG-003, Page 3

Current behaviour:

When the user taps the three-dot menu, the action menu opens, but part of it is hidden behind or inside the project card/image. The menu appears tucked into the project image, making its options partially obscured.

Expected behaviour:

The complete action menu should appear clearly above the project card, with all options visible and accessible.

Required correction:

Correct the menu's positioning, stacking order, overflow and container behaviour so that the project card or image does not cover or clip the menu.

Acceptance criteria:

All four options are fully visible: Edit, Download, Save as Template and Delete.

The menu is not clipped by the project card or image.

The menu remains properly positioned on mobile and desktop.

All four actions remain functional.

Correcting the menu does not interfere with other project-card interactions.

Part B: Existing feature enhancements

ENHANCEMENT-002 — Complete project-to-template management functionality

Feature/location: Project action menu and Upload Sheet template selection

PDF Visual Reference: ENHANCEMENT-002, Page 3

Required functionality:

1. Save as Template

Add a functional option that allows users to save an existing project/design as a reusable template.

The saved template must appear in the template selection list when the user uses Upload Sheet.

Selecting the saved template must allow the existing auto-population process to work with it.

Saving a project as a template must not remove the original project from the Projects list.

2. Remove from Template

Provide an option to remove a previously saved template from the available template collection.

Once removed, it must no longer appear in the template selection list under Upload Sheet.

The original project must remain available in Projects.

Removing a template must not accidentally delete the original project.

3. Keep project deletion and template deletion independent

These are two different operations.

Delete project ensures only Project is deleted; its separately saved template remains available.

Remove from Template ensures Template is removed from the template collection while original project remains.

Delete project ensures project is deleted, if project has been made a template, template remains. If template has already been removed, both (project and template) are gone because the user explicitly performed both operations.

Important implementation requirement:

A project saved as a template must have independently managed template availability. Deleting the original project must not automatically delete its saved template. Likewise, removing a template must not delete its original project.

Acceptance criteria:

Projects can be saved as reusable templates.

Saved templates appear in Upload Sheet's template selector.

Selecting a template supports the existing auto-population workflow.

Users can remove templates independently.

Project deletion does not silently delete a saved template.

Template removal does not silently delete a project.

Existing project editing, downloading and saving functionality remains intact.

The system checks whether the project has actually been saved as a template.

If it has not, display the specified error message.

Do not delete the project or modify the template collection.

If it has, allow the normal Remove from Template operation.

4. Validation when removing a template that has not been saved

If a user selects Remove from Template for a project that has not previously been saved as a template, the system must not perform any deletion.

ENHANCEMENT-003 — Replace emoji action representations with SVG icons

Feature/location: Project action menu

PDF Visual Reference: ENHANCEMENT-003, Page 3

Required correction:

Use proper SVG icons to represent the following project actions:

Edit

Download

Save as Template

Delete

Design requirements:

Use actual SVG symbols/icons, not emojis.

Icons must be visually clear and recognizable.

Maintain consistent sizing, alignment and visual styling.

Each icon must correspond correctly to its action.

Icons must not replace or interfere with the functionality or text labels of the actions.

BUG-004 — Remove redundant design-number navigation tab

Feature/location: Design editor, top navigation bar

PDF Visual Reference: BUG-004, Page 4

Current behaviour:

A long, horizontally oval navigation tab appears at the top centre of the interface. It contains left and right arrowheads and a design counter such as:

Design 2 of 8

Design 3 of 8

Design 4 of 8

Expected behaviour:

This design-number tab should not appear and should be removed

Required correction:

Remove the oval design-number navigation tab, including its arrow controls.

Reason:

The application already provides thumbnail navigation and other ways to move between designs. This additional navigation tab is redundant and occupies unnecessary interface space.

Acceptance criteria:

The oval design-number tab is removed.

Existing thumbnail navigation remains functional.

Existing design navigation controls elsewhere remain unaffected.

BUG-005 — Add Horizontal Swipe Navigation Between Designs

Feature/location: Design editor, bottom of design workspace

PDF Visual Reference: BUG-005, Page 5

Current Behaviour:

The user cannot conveniently move between designs by swiping horizontally across the design workspace.

Expected Behaviour:

The user should be able to move from one design to the next by horizontally swiping with a thumb or finger.

Required Correction:

Implement horizontal touch-swipe navigation between designs, with three small circular page indicators positioned at the bottom of the design workspace.

The indicators should communicate the current navigation position and respond appropriately as the user moves between designs.

Acceptance Criteria:

1. Users can swipe horizontally to move to the next or previous design.
2. The three circular indicators appear at the bottom of the design workspace.
3. The indicators update to reflect the current position.
4. Navigation works on mobile touchscreens.
5. Existing thumbnail navigation continues to work.

6. Swiping between designs does not accidentally interfere with editing, selecting or moving objects on the canvas.

Important Technical Distinction:

The three circles must not be confused with the four unnecessary dots identified in BUG-002.

- **BUG-002:** Remove the unwanted four-dot indicator at the bottom of the general page.
- **BUG-005:** Introduce the intended three-circle navigation indicator specifically for moving between designs.

These components serve different purposes and must be implemented independently. The three circles must not imply that the project contains only three designs. They should correctly represent navigation across however many designs are available.

Additional Technical Clarification — Three-Circle Navigation Indicator

The three circular indicators are not intended to represent the total number of designs in a project. They are a fixed three-indicator navigation aid that must work regardless of whether the project contains 3, 8, 15 or more designs.

Required behaviour:

1. Always display exactly three small circular indicators.
2. Use the indicators to communicate the user's relative position within the complete sequence of designs.
3. As the user swipes through designs, update the active indicator to reflect their position.
4. For projects containing more than three designs, the indicators must continue to represent progress through the entire design sequence, not stop at the third design.
5. The swipe navigation must continue working until the user reaches the first or last design.
6. The three indicators must remain synchronized with thumbnail navigation and any other existing design-navigation method.

Implementation note: Must choose a consistent position-mapping approach for the three indicators so that all designs are represented accurately, including projects containing more than three designs. Do not create one indicator per design or limit navigation to three designs.

BUG-006 — Text Tab Is Nonfunctional and Its Interface Is Incorrectly Assigned to the Plus Tab

Feature/location: Design editor, main toolbar, Text tab and Add (+) tab

PDF Visual Reference: BUG-006, Page 6

Current Behaviour:

1. The Text tab, identified by the letter A, does not respond when tapped.
2. Tapping the Text tab produces no popup, panel or other interface.
3. The Add (+) tab currently opens a popup containing the interface and functionality intended for the Text tab.
4. Consequently, the Text tab is nonfunctional, while the Add tab has been assigned the wrong interface.

Expected Behaviour:

The Text tab and Add (+) tab must each open their respective interfaces and perform their intended functions.

Required Correction:

1. Restore the functionality of the Text tab.
2. Move the existing text-related popup interface currently assigned to the Add (+) tab to the Text tab.
3. Ensure that tapping the Text tab opens the correct text-editing and text-related interface.
4. Correct the Add (+) tab so that it no longer opens the text-related popup. It must retain its own intended Add functionality.
5. Ensure that both tabs operate independently and open the correct interfaces when tapped.

Acceptance Criteria:

1. Tapping the Text tab opens its text-related popup or panel.
2. The existing text-related popup functionality is accessible through the Text tab.
3. Tapping the Add (+) tab opens its intended Add interface, not the Text interface.
4. Neither tab is inactive or incorrectly assigned.
5. The two tabs do not trigger each other's interfaces.
6. Existing text-related functionality is preserved when transferring the popup to the correct tab.
7. Existing Add (+) functionality is preserved or correctly restored.
8. Both tabs work correctly on mobile and desktop.
9. Correcting the tab assignments does not interfere with unrelated editor features.

Important Implementation Instruction:

Do not simply duplicate the popup across both tabs. Correct the underlying tab-to-interface assignments so that each tab invokes its own intended functionality. Inspect existing event handlers, component assignments and dependencies before making changes. Preserve working functionality and avoid unnecessary rewrites.

TEXT FEATURE SPECIFICATION — Text Popup: Text Insertion Functions

Feature/location: Design editor → Text (A) tab

PDF Visual Reference: Page 7

1. Add Text Box

Purpose:

Allows the user to insert a normal, editable text box into the design currently being worked on.

Expected Behaviour:

1. When the user taps **Add Text Box**, the application inserts a new text box into the active design.
2. The text box must use a normal paragraph-style text format by default.
3. The user can select the text box and type any content they want.
4. The text box must be editable and behave like a normal text element.
5. The newly inserted text box must belong only to the currently active design by default.
6. It must not automatically appear on other designs or pages in the project.

Design-specific behaviour:

If the user is currently editing Design 3, tapping Add Text Box must insert the text box into Design 3 only.

The user can subsequently use the application's Sync feature to synchronize the text box across other designs, where supported by the existing synchronization functionality.

Important: Adding a text box and synchronizing a text box are two separate operations. Synchronization must not happen automatically when a text box is first inserted.

2. Add a Heading

Purpose:

Allows the user to insert a heading text element into the active design.

Expected Behaviour:

When tapped, the application inserts a heading-style text element into the currently active design.

3. Add a Subheading

Purpose:

Allows the user to insert a subheading text element into the active design.

Expected Behaviour:

When tapped, the application inserts a subheading-style text element into the currently active design.

4. Add Body Text

Purpose:

Allows the user to insert a body-text element into the active design.

Expected Behaviour:

When tapped, the application inserts a body-text-style element into the currently active design.

Important Functional Distinction

These four options must not be treated as interchangeable.

- **Add Text Box:** Inserts a normal paragraph-style editable text box for freely entering content.
- **Add a Heading:** Inserts a heading-style text element.
- **Add a Subheading:** Inserts a subheading-style text element.
- **Add Body Text:** Inserts a body-text-style element.

Scope rule: All four insertion actions apply only to the currently active design by default. They must not automatically insert elements across every design. Synchronization is a separate operation.

5. Add Page Number

Purpose:

Allows the user to automatically insert sequential page numbers across all designs/pages in the current project.

Expected Behaviour:

When the user taps **Add Page Number**, the application should automatically insert page numbers across every design in the current project, starting from the first design and continuing sequentially to the last.

Numbering sequence example:

If the project contains 11 designs, the application should insert:

- Design 1: 1
- Design 2: 2
- Design 3: 3
- Design 4: 4
- Design 5: 5
- Design 6: 6
- Design 7: 7
- Design 8: 8
- Design 9: 9
- Design 10: 10
- Design 11: 11

Placement:

- Page numbers must be positioned at the lower-right corner of every design by default.
- The number must be inserted as an editable text element.
- Each number must belong to its corresponding design.

User Control:

After automatic numbering, the user must be able to select an individual page number and delete it if they do not want numbering displayed on that particular design.

For example, if the user does not want a page number on Design 1, they can select and delete the number 1 without automatically deleting the numbers on the other designs.

Important Functional Requirements:

1. Page numbering must be a project-wide insertion operation, not a single-page insertion operation.
2. Numbering must begin at 1 on the first design.
3. Each subsequent design must receive the next sequential number.
4. The numbering must cover every design in the project.
5. The inserted numbers must remain individually selectable and editable.
6. Deleting one page number must not delete the other page numbers.
7. The default placement must be the lower-right corner of each design.
8. Adding page numbers must not replace, overwrite or interfere with existing text elements.

Important Distinction:

Add Page Number is different from Add Text Box and other additional elements on the pop up for the A Tex tab.. Add Text Box inserts one text element into the currently active design. Add Page Number automatically inserts sequential numbers across all designs in the project.

6. Add Table to Page

Purpose:

Allows the user to insert a customizable table directly into the currently active design.

Expected Behaviour:

When the user taps **Add Table to Page**, the application must display a popup requesting the number of rows and columns for the table.

Table creation process:

1. The user taps Add Table to Page.
2. A popup appears with two input fields:
    - Number of Rows
    - Number of Columns
3. The user enters the desired number of rows and columns.
4. The user taps OK.
5. The application automatically generates the table and inserts it into the currently active design.

Table Appearance:

- The table must have medium-weighted borders.
- Border lines must be neither excessively thick nor excessively thin.
- Maintain a clear, balanced and professional table appearance.
- Table cells must be visibly separated by the borders.

Table Editing and Customization:

After insertion, the user must be able to:

1. Enter text or other supported content into the table cells.
2. Increase the overall table size.
3. Reduce the overall table size.
4. Adjust the table to fit the design.
5. Select and interact with the table as an editable design element.

Scope of Insertion:

The table must be inserted only into the currently active design.

For example, if the user is working on Design 4, the table must appear on Design 4 only. It must not automatically appear on Designs 1, 2, 3 or any other design.

Download and Export Preservation Requirement:

All tables inserted into designs must be included in exported/downloaded files.

The application's export functionality must preserve every element and modification made to the design, including:

- Tables and their contents.
- Text boxes and text formatting.
- Headings, subheadings and body text.
- Images and other visual elements.
- Templates and design elements.
- All positioning, sizing and arrangement adjustments.

The exported file must accurately reproduce the completed design as displayed in the editor.

Important: There must be no incomplete or selective export in which a design downloads but its tables, images, text or other supported elements are missing.

Acceptance Criteria:

1. Tapping Add Table to Page opens the row-and-column input popup.
2. The user can specify the desired number of rows and columns.
3. Tapping OK generates the requested table.
4. The generated table appears on the currently active design only.
5. Table borders have a balanced medium weight.
6. The user can enter content and resize the table.
7. The table remains editable within the design editor.
8. All table content, borders, dimensions and positioning are preserved during export.
9. Exported designs include all other supported elements and modifications without omissions.

7. Add Horizontal Line

Purpose:

Allows the user to insert a simple, adjustable horizontal line into the currently active design.

Expected Behaviour:

When the user taps **Add Horizontal Line**, the application automatically inserts a horizontal line into the current design.

Line Appearance:

- The line must have a medium thickness.

- It must not be excessively thick or too thin.
- It should have a clean, simple and professional appearance.
- The line must be horizontally aligned by default.

Line Resizing and Controls:

The inserted horizontal line must have two circular control handles, one at each endpoint.

The user must be able to:

1. Touch and drag either endpoint to increase the line's length.
2. Drag either endpoint inward to decrease its length.
3. Adjust the line's length freely using the two endpoint handles.
4. Keep the line editable after insertion.

The interaction should follow the familiar endpoint-dragging behaviour used by Canva's line elements.

Scope of Insertion:

The horizontal line must be added only to the currently active design. It must not automatically appear on other designs unless the user separately uses the Sync functionality.

Acceptance Criteria:

1. Tapping Add Horizontal Line inserts a horizontal line into the active design.
2. The line has a balanced medium thickness.
3. Two circular control handles appear at its endpoints.
4. Dragging either handle increases or decreases the line's length.
5. The line remains selectable and editable.
6. Existing design elements and editing functionality remain unaffected.
7. The line is preserved correctly during download and export.

Additional Clarification for Section 6 — Add Table to Page

Table Resizing Controls:

The generated table must have four circular resizing handles positioned at its four corners.

The user must be able to:

1. Drag the corner handles outward to increase the table's overall size.
2. Drag the corner handles inward to reduce its overall size.
3. Resize the table interactively using touch or mouse input.
4. Preserve the table's contents and structure while resizing.

The four corner handles must remain accessible and must not interfere with entering or editing text inside the table.

ENHANCEMENT — Font Combinations: Functional Presets and Custom Upload Library

Feature: Font Combinations
Enhancement Type: Improvement to an existing feature (See Page 8 of visual reference pdf)

1. Current Behaviour

The Font Combinations section currently displays three options:

- Aesthetic
- Now Open
- Bold Moves

However, tapping these options does not produce any action. They are currently non-functional.

2. Required Changes

A. Create Two Default Font Combinations

Litdom Batch Studio should create and provide two ready-to-use sample font combinations.

The first combination should appear in the first existing box, labelled **Aesthetic**.

The second combination should appear in the second existing box, labelled **Now Open**.

Each combination must contain a properly defined pairing of compatible fonts, with the appropriate font assignments for headings, subheadings and body text.

These combinations must be functional. Tapping either one should activate its font combination rather than simply displaying the option.

B. Replace Bold Moves with an Upload Feature

Remove the existing **Bold Moves** option and replace it with:

Upload Your Font Combination

When tapped, this option should allow the user to import their own font combinations into Litdom Batch Studio.

C. Supported Upload Formats

The upload system should support:

1. **JSON (.json):** For importing a font-combination configuration. This file should describe the combination's name, font assignments and relevant styling settings.
2. **TTF (.ttf):** For importing actual font files.
3. **OTF (.otf):** For importing actual font files.

A JSON configuration may reference fonts already available in the application or fonts supplied separately by the user.

Where a combination requires custom fonts, the application should allow the user to upload the necessary TTF or OTF files as well.

The application should validate uploaded files and provide a clear error message if a file is unsupported, invalid or incomplete. It must not silently replace a missing custom font with an unrelated font.

D. Display Uploaded Combinations

After a successful upload, the imported combination should appear in the Font Combinations section as a new, selectable font-combination option.

The uploaded combination should have its own identifiable name.

The original two default combinations, Aesthetic and Now Open, should remain available.

The upload option should also remain available so that the user can add more combinations.

E. Persistent Font Combination Library

Uploaded font combinations must be retained for future use.

Once a user imports a combination, it should remain available whenever they return to Litdom Batch Studio, including when they:

- Start a new project.
- Open another existing project.
- Refresh or reopen the application.
- Begin another design session.

The application should not require the user to upload the same combination repeatedly.

Custom font files associated with an uploaded combination must also be retained and remain accessible to that combination.

F. Functional Application of Font Combinations

Every displayed font combination must be interactive.

When a user selects a combination, the application must apply the fonts assigned by that combination rather than merely highlighting the selected box.

The application must correctly identify and use the relevant font assignments for headings, subheadings and body text.

The selected combination must use the actual uploaded or available fonts, not substitute fonts without informing the user.

3. Acceptance Criteria

- Two functional default combinations are available: Aesthetic and Now Open.
- Bold Moves is replaced with Upload Your Font Combination.
- Tapping either default combination performs its intended font-application action.
- Tapping the upload option opens a working file-import process.
- JSON configurations can be imported.
- TTF and OTF font files can be imported for use by combinations.
- Successfully uploaded combinations appear as additional selectable options.
- Uploaded combinations remain available across future sessions.
- Custom font files remain associated with their respective combinations.
- Invalid or incomplete uploads produce understandable error messages.
- Existing project content and unrelated font functionality remain intact.

Current-Design-Only Rule

Selecting and applying any font combination must affect only the currently active design. It must not automatically change fonts or formatting on other designs in the same project.

All design modifications in Litdom Batch Studio must follow this rule by default. Changes should affect other designs only when the user deliberately invokes the Sync Layout feature.

Sync Layout is a separate operation and must not be triggered automatically when applying a font combination.

Protected Existing Features — Mandatory Preservation Rule

The existing Add Heading, Add Subheading and Add Body Text features are working correctly.

Do not modify, rewrite, replace, restructure or interfere with their existing implementation, functionality, styling or behaviour while implementing the Font Combinations enhancement or Sync Layout.

Font Combinations must work with the existing design elements without breaking their insertion, editing or formatting functionality.

If integration requires interaction with any of these protected features, inspect the dependencies first and implement the integration without changing their existing working behaviour.

After implementation, verify that all three features still function exactly as they did before the changes.

BUG-007 — Plus (+) Button Lacks Shape-Insertion Functionality

Feature/location: Plus (+) / Add button
PDF Visual Reference: BUG-007, Page 9

Current Behaviour

The Plus (+) button previously displayed a popup whose functionality has already been transferred to the Text (A) tab.

The Plus button now needs its own clearly defined functionality. It should serve as the shape-insertion tool.

Expected Behaviour

When the user taps the Plus (+) button, a simple shape-selection panel should appear.

Required Correction

1. Shape Selection

The panel should provide basic shapes, including:

- Rectangle
- Square
- Circle
- Triangle
- Star
- Other common basic shapes, where supported.

2. Shape Insertion and Editing

When the user selects a shape:

- Insert it into the currently active design.
- Allow the user to select and move it.
- Allow the user to resize it.
- Allow the user to change its fill colour and border styling, where supported.

3. Current-Design-Only Rule

Adding a shape must affect only the currently active design.

Other designs in the project must remain unchanged unless the user deliberately uses Sync Layout.

4. Functionality Preservation

The previous popup has already been transferred to the Text (A) tab. Do not move it back to Plus or duplicate it.

Do not modify or interfere with the Text tab, its popup, or its existing working text-insertion functions.

Keep the Plus button focused on shape insertion. Do not introduce unrelated tools.

Acceptance Criteria

- Tapping Plus opens the shape-selection panel.
- Selecting a shape inserts it into the active design.
- Shapes can be selected, moved and resized.
- Supported fill and border settings work.
- Other designs remain unchanged unless Sync Layout is deliberately used.
- Existing Text tab functionality remains intact.

Shape Default Colour

Every newly inserted shape must have a solid black fill by default (#000000).

All shapes must initially appear in black, regardless of their shape type.

Do not assign random colours, gradients or other default colours.

BUG-008 — Uploaded Images Are Added as New Pages Instead of Design Elements

Feature/location: Upload File / Image Upload
PDF Visual Reference: BUG-008, Page 10

Current Behaviour

When the user uploads an image from their phone, the application treats the uploaded image as a new page instead of inserting it into the currently active design.

Expected Behaviour

The Upload feature should allow users to import images directly from their phones and place them onto the currently active design as image elements.

The uploaded image should appear over the existing design content, just like adding an image to a page in Canva.

Required Correction

1. Upload Images from the Phone

When the user taps Upload, allow them to select an image from their phone’s files or image gallery.

2. Insert Image into the Current Design

After the user selects an image:

- Insert the image into the currently active design.
- Display it as an overlay/image element on the existing design.
- Preserve all existing text, shapes, tables and other design elements.
- Do not create a new design or page for the uploaded image.
- Do not replace the existing page background or existing design content.

3. Image Adjustment

The inserted image must be selectable and editable.

Allow the user to:

- Move the image around the design.
- Resize it to the desired dimensions.
- Adjust its position to fit the design.
- Place it alongside or over other design elements.

The image should remain an independent design element so that the user can adjust it without unintentionally modifying other content.

4. Current-Design-Only Rule

An uploaded image must be inserted into the currently active design only.

It must not automatically appear on other designs in the project unless the user deliberately uses Sync Layout.

5. Complete Export Functionality

When the user exports or downloads their designs, the exported output must include all design elements and changes as displayed in the application.

This includes:

- Uploaded images.
- Text and font formatting.
- Shapes.
- Tables and their contents.
- Horizontal lines.
- Element positions and dimensions.
- Other supported design elements.

The application must not selectively omit uploaded images or other elements during export.

Acceptance Criteria

- The user can select images from their phone.
- Uploaded images appear on the active design as editable image elements.
- Uploaded images do not create new pages.
- Existing page content remains intact.
- Images can be moved and resized.
- Other designs remain unchanged unless Sync Layout is deliberately used.
- Exported designs contain the uploaded images and all other visible design elements.
- Existing DOCX-to-template mapping and other working features remain unaffected.

BUG-009 — Sync Layout and Sync Text Field Functions Are Nonfunctional and Apply Incorrect Formatting

Feature/location: Sync Layout
PDF Visual Reference: BUG-009, Page 11

1. CURRENT BEHAVIOUR

The existing Sync Layout feature is not functioning correctly. It contains two intended operations:

1. Sync This Design's Layout.
2. Sync Text Fields Layout.

However, neither function currently works reliably.

When the user designs an element, such as a title, and attempts to synchronise its formatting across other designs, the application sometimes applies incorrect formatting.

Example of the existing problem:

The source title has:

- Yellow text.
- A blue outline.

- A particular font size.
- Specific font weight and effects.

After synchronisation, the destination titles become larger and red instead of adopting the original yellow colour and blue outline.

This changes and damages existing designs instead of synchronising them accurately.

There are also interface problems:

- The two synchronisation functions are not clearly separated.
- There are no proper, dedicated final action buttons for both operations.
- Custom page selection does not properly display the actual destination pages.
- Users cannot reliably identify which pages they are selecting.
- The current interface does not provide a complete, dependable synchronisation workflow.

2. REQUIRED CORRECTION — TWO INDEPENDENT SYNCHRONISATION FUNCTIONS

The Sync Layout feature must contain two separate sections.

They must have different purposes, independent settings and separate final action buttons.

Do not combine them into one operation.

PART A — SYNC THIS DESIGN'S LAYOUT

3. Purpose

This function synchronises the overall visual layout and formatting of the currently active source design across the destination pages selected by the user.

The currently active design serves as the source design.

The destination designs should adopt the source design's visual arrangement and formatting.

This includes:

- Title and heading appearance.
- Font family and font size.
- Font weight, including bold and italic.
- Text colours.

- Text outlines and effects.
- Text alignment.
- Text positioning.
- Subtitle and body-text styling.
- CTA appearance and styling.
- Element sizes and spacing.
- Other supported design-element formatting and arrangement.

Example:

Suppose the source design contains:

- A yellow title with a blue outline.
- A particular font family and font size.
- A specific title position.
- A styled subtitle.
- A CTA with a particular colour, size and arrangement.

When the user synchronises the source layout, the destination designs should adopt those same visual properties.

The source design must be treated as the reference. The application must reproduce its actual formatting values rather than approximate them.

Important content-preservation rule

Synchronising the design layout must not indiscriminately replace page-specific wording, mapped data or other unique content.

The destination pages should adopt the source design's layout and visual styling while retaining their individual content and data.

The objective is to make the designs look consistent, not to turn every page into a duplicate of the source page's written content.

4. Interface for Sync This Design's Layout

Create a dedicated section labelled:

SYNC THIS DESIGN'S LAYOUT

This section must contain the following controls.

A. Destination Page Selection

Provide three options:

1. This Page
2. All
3. Custom (0 Selected)

This Page

This should be the default selection.

It applies the operation to the currently active page only. No other page should be modified.

All

This allows the user to apply the source design's layout and formatting across all pages in the current project.

Custom (0 Selected)

This allows the user to choose specific destination pages.

When Custom is selected, display the available project pages as identifiable visual thumbnails.

The user must be able to:

- View the available page thumbnails.
- Select individual destination pages.
- Select multiple pages.
- Deselect previously selected pages.
- Clearly identify which thumbnails are selected.
- See the total number of selected pages.

For example, if the user selects pages 1, 2 and 3, the interface should display those three pages as selected and indicate that 3 pages have been selected.

Do not simply display a number without showing the actual pages available for selection.

B. Final Synchronisation Button

Below the destination-page selection, provide a dedicated button labelled:

SYNC DESIGNS LAYOUT

When tapped, the application must execute the complete design-layout synchronisation using the selected destination-page scope.

It must not execute the operation merely because the user selected a destination scope.

The user must deliberately press the final button.

PART B — SYNC TEXT FIELDS LAYOUT

5. Purpose

This function synchronises only the visual appearance and formatting of selected text fields across the chosen destination pages.

Unlike Sync This Design's Layout, this function must not synchronise the entire design.

The user chooses which text field or fields should receive consistent formatting.

Examples include:

- Title.
- Heading.
- Subtitle.
- Body Text.
- CTA.
- Other supported text fields.

Example

Suppose the user wants all CTA elements across the project to have the same appearance.

The user selects the CTA field from the currently active design.

The application should synchronise the CTA's:

- Font family.
- Font size.
- Font weight.
- Text colour.
- Outline.
- Effects.
- Alignment.
- Position.
- Other applicable formatting properties.

However, the other text fields must remain unchanged.

The titles, subtitles and body text on destination pages must retain their existing appearance and formatting.

MANDATORY: PRESERVE THE ORIGINAL TEXT CONTENT

This function must synchronise appearance and formatting only.

It must NEVER:

- Copy the source text content into destination pages.
- Replace destination wording.
- Overwrite mapped text.
- Delete existing text.
- Change individual page-specific content.

Example:

Page 1 CTA: Start Your Journey

Page 2 CTA: Download the Guide

Page 3 CTA: Read More

After synchronising the CTA field, each page must retain its original wording.

All three CTA elements should simply adopt the same visual appearance and formatting.

The same rule applies to titles, subtitles, headings, body text and every other supported text field.

Only appearance and formatting should synchronise. Text content and mapped data must remain independent on every page.

6. Interface for Sync Text Fields Layout

Create a separate section labelled:

SYNC TEXT FIELDS LAYOUT

This section must contain the following controls.

A. Select Fields

Provide a field-selection area labelled:

SELECT FIELDS

Display the available text fields from the currently active design.

Allow the user to select the individual field or fields they want to synchronise.

The user must be able to select one field or multiple fields.

Clearly indicate which fields have been selected.

Do not automatically select every text field.

B. Sync With

Below the field-selection area, provide a destination selector labelled:

SYNC WITH

It must contain three options:

1. This Page
2. All Pages
3. Custom

This Page

This should be the default selection.

It applies the operation to the currently active page only.

All Pages

Synchronise the selected text fields' appearance and formatting across all pages in the current project.

Custom

Allow the user to select specific destination pages within the current design or project.

When Custom is selected, display the available project pages as visual thumbnails.

The user must be able to:

- View all available page thumbnails.
- Select individual pages.
- Select multiple pages.
- Deselect pages.
- Clearly identify selected pages.
- See the total number of selected pages.

For example, if the user selects pages 2, 4, 5 and 7, those exact pages must be identified as selected.

Do not merely display a selection count without providing access to the actual page-selection interface.

C. Apply Button

After the user has selected their custom destination pages, provide a button labelled:

APPLY

The Apply button should confirm the selected destination pages.

After applying, the selected pages and selection count must remain accurately reflected in the interface.

Applying the selection must not execute synchronisation yet.

D. Final Synchronisation Button

After the field and destination selections, provide a dedicated button labelled:

SYNC TEXT FIELD

When tapped, execute the selected text-field synchronisation.

Only the selected text fields should be affected.

Only their visual appearance and formatting should be synchronised.

Their original text content and mapped data must remain unchanged.

PART C — FORMATTING ACCURACY

7. Exact Formatting Preservation

This is a critical requirement for both synchronisation functions.

The application must retrieve and reproduce the actual formatting properties of the source design or selected text field.

It must preserve:

- Exact colours.
- Font families.
- Font sizes.
- Font weights.
- Text outlines.
- Text effects.
- Alignment.
- Positioning.
- Dimensions.
- Spacing.
- Other supported styling properties.

For example, if the source title is yellow with a blue outline, the destination title must also be yellow with a blue outline.

Do not change it to red.

Do not arbitrarily increase or decrease font sizes.

Do not introduce random colours, effects or default formatting.

Do not approximate the source styling when the actual formatting values are available.

The application must use the existing source element's actual styling information.

8. Synchronisation Scope and Current-Design-Only Rule

All ordinary design modifications in Litdom Batch Studio must affect the currently active design only by default.

This includes changes to text, fonts, shapes, tables, lines, images, formatting and positioning.

Synchronisation is an intentional exception.

Changes may propagate to other pages only when the user deliberately chooses the destination scope and presses the appropriate final Sync button.

Both synchronisation functions must default to This Page.

Selecting All or Custom must never happen automatically.

The two synchronisation functions must maintain their independent settings and must not accidentally execute one another.

9. Functional Requirements

The completed feature must support the following workflows.

Workflow A: Synchronise the complete design layout

1. Open Sync Layout.
2. Select Sync This Design's Layout.
3. Choose This Page, All or Custom.
4. If Custom is selected, choose the required destination pages using their thumbnails.
5. Press Sync Designs Layout.
6. Apply the source design's layout and formatting to the selected destination pages.

Workflow B: Synchronise selected text fields

1. Open Sync Layout.
2. Select Sync Text Fields Layout.
3. Choose the text field or fields to synchronise.
4. Select This Page, All Pages or Custom.
5. If Custom is selected, choose the destination pages using their thumbnails.
6. Press Apply to confirm the custom page selection.
7. Press Sync Text Field.
8. Apply only the selected fields' visual formatting to the destination pages.
9. Preserve all original text content and mapped data.

10. Functionality Preservation and Non-Regression

Before implementation, inspect the existing Sync Layout architecture, the design-element data structures, formatting properties, page-selection logic and synchronisation mechanisms.

Do not perform a superficial visual redesign while leaving the underlying functionality broken.

Implement both synchronisation operations so that they work end to end.

Preserve the following existing working features:

- DOCX-to-template mapping.
- Add Heading.
- Add Subheading.
- Add Body Text.
- Existing text insertion and editing.
- Existing design navigation.
- Existing page-specific content.
- Existing project and template functionality.

Do not rewrite, replace or interfere with unrelated features.

If shared components or dependencies need to be modified, inspect them first and ensure that the changes do not break existing functionality.

If a correction cannot be implemented safely without affecting an existing working feature, identify the conflict before proceeding.

11. COMPREHENSIVE TESTING AND ACCEPTANCE CRITERIA

Do not consider this bug fixed merely because the interface appears correct.

Both synchronisation functions must be tested independently.

Test A: Sync This Design's Layout

- Create a source design with a yellow title and blue outline.
- Give it a specific font size, font weight, alignment and position.
- Include a subtitle and a styled CTA.
- Synchronise with All Pages.
- Verify that the destination pages adopt the source layout and formatting.
- Verify that no arbitrary colours, font sizes or effects are introduced.
- Verify that page-specific wording and mapped data are preserved.

Repeat the test using Custom Selected Pages.

Verify that only the selected pages are modified.

Test This Page and verify that no other page changes.

Test B: Sync Text Fields Layout

- Create destination pages with different titles, subtitles and CTAs.
- Give their CTA fields different colours, font sizes, outlines and effects.
- Select only the CTA field from the source design.
- Synchronise with All Pages.
- Verify that all CTA fields adopt the source CTA's appearance and formatting.
- Verify that each CTA retains its original wording.
- Verify that titles, subtitles and other unselected fields remain unchanged.

Repeat the test using Custom Selected Pages.

Verify that only the selected destination pages and selected fields are modified.

Test This Page and verify that no other page changes.

Test C: Interface and Selection

Verify that:

- Both sections appear separately.
- Each section has its own destination selector.
- Custom displays actual page thumbnails.
- Multiple pages can be selected and deselected.
- Selected thumbnails are visually distinguishable.
- The selected-page count updates correctly.
- Apply confirms custom page selection.
- The two final Sync buttons work independently.
- Selecting an option does not execute synchronisation prematurely.

Test D: Regression Testing

After implementation, verify that:

- DOCX-to-template mapping still works.
- Add Heading still works.
- Add Subheading still works.
- Add Body Text still works.
- Existing text content remains editable.
- Existing page navigation works.
- Other project functionality remains intact.
- No unrelated design content is damaged.

12. FINAL IMPLEMENTATION REQUIREMENT

Implement the complete Sync Layout feature, including its interface, page-selection mechanism, formatting retrieval, synchronisation logic and final action buttons.

Do not leave placeholder buttons, decorative selections or incomplete workflows.

Do not treat a visually updated interface as a successful correction if the underlying synchronisation still fails.

Both functions must work reliably with All Pages, Custom Selected Pages and This Page.

The completed feature must accurately synchronise the intended visual formatting, preserve page-specific content, and avoid damaging unrelated design elements.

The objective is reliable, precise and predictable synchronisation, not merely making the Sync Layout interface look functional.

BUG-010 — Download Icons and Export Quality/Completeness Issues

Feature/location: Download / Export Section
PDF Visual Reference: BUG-010, Page 12

1. CURRENT BEHAVIOUR

The Download section has two areas requiring correction.

A. Incorrect File-Type Icons

The application currently uses emojis or unsuitable image representations for the available export formats, including:

- PNG
- JPG
- WebP
- PDF

The file-type dropdown also uses these unsuitable representations.

The current appearance is not sufficiently clean or professional.

B. Export Quality and Completeness

The export functionality must reliably preserve the complete appearance and quality of designs created in Litdom Batch Studio.

Every supported design element must be included in the downloaded output.

This includes uploaded images, text, tables, horizontal lines, shapes and other elements added to the design.

The exported result must not omit elements, distort their appearance, reduce their quality unnecessarily or change their arrangement.

2. REQUIRED CORRECTION A — USE PROPER SVG FILE-TYPE ICONS

Replace all emoji-based and unsuitable file-type representations with clean, professionally designed SVG icons.

Provide appropriate SVG icons for:

- PNG
- JPG
- WebP
- PDF

Apply the correct icons throughout the Download section, especially inside the file-type dropdown.

Requirements:

- Use actual SVG vector icons, not emojis.
- Use an appropriate visual representation for each file format.
- Maintain consistent icon dimensions, alignment and visual styling.
- Ensure icons remain sharp at different screen sizes.
- Keep the file-format labels visible and readable.
- Ensure the dropdown looks clean and professional on both mobile and desktop.

Do not use decorative emoji substitutes.

3. REQUIRED CORRECTION B — HIGH-QUALITY EXPORTS

The download system must produce clean, high-quality exports that faithfully reproduce the finished design.

The output should be suitable for publishing and use on the user's platforms.

A. Preserve Design Resolution and Quality

Maintain high export resolution and image quality.

Do not unnecessarily compress, blur, pixelate or degrade exported designs.

Use appropriate rendering and export settings for each supported file format.

Where resolution settings are available, provide suitable high-quality output options without compromising the default export quality.

B. Preserve Every Design Element

The exported design must contain every supported element visible in the completed design.

This includes:

- Titles and headings.
- Subheadings and body text.
- Text boxes and their formatting.
- Font families, sizes and weights.
- Text colours, outlines and effects.
- Tables, including their contents, borders and dimensions.
- Horizontal lines and other shapes.
- Uploaded images.
- Images positioned as overlays.
- Element positions, sizes and alignment.
- Backgrounds and other supported design elements.
- Other supported visual modifications made by the user.

Nothing should be selectively omitted during export.

C. Preserve Formatting and Appearance

The downloaded result must closely match the design displayed in the editor.

Preserve:

- Original colours.
- Font styling.
- Text alignment.
- Outlines and effects.
- Table borders.
- Shape dimensions.
- Line thickness.
- Image proportions.
- Element positioning.
- Spacing and layout.
- Other supported visual properties.

Do not unexpectedly change the arrangement or appearance of elements during rendering.

D. Preserve Uploaded Images

Images imported from the user’s phone must be included in the exported design.

Preserve their visual quality, proportions and positioning.

Do not omit uploaded images simply because they were inserted as independent overlay elements rather than backgrounds.

E. Complete Design Export

Export must capture the complete finished design, not just selected portions of it.

All supported elements must be rendered together as they appear in the editor.

The export process must account for the complete design composition, including elements that overlap or sit above other elements.

Ensure that the exported output does not contain missing objects, unexpected clipping, misplaced elements or incomplete rendering.

4. REQUIRED CORRECTION C — DOWNLOAD DIRECTLY TO THE USER'S DEVICE

The user must be able to download completed designs directly to their device storage.

On mobile devices, the download process should work with the available browser or operating-system download mechanism so the user can save the exported files to their phone.

On desktop devices, users should similarly be able to save exported files to their computer.

Do not merely generate an export preview without providing a working download action.

The downloaded files must be accessible to the user for subsequent use on their publishing and social platforms.

5. FORMAT-SPECIFIC REQUIREMENTS

Preserve the intended output format when exporting.

- **PNG:** Produce a clean, high-resolution image export.
- **JPG:** Produce a high-quality image export without unnecessary compression.
- **WebP:** Preserve image clarity and the intended design appearance.
- **PDF:** Preserve the complete design composition across the exported pages, including text, tables, shapes, lines and uploaded images.

Where possible, use vector rendering for text and vector elements in PDF exports to maintain sharpness. Raster images should retain their available source quality.

6. FUNCTIONALITY PRESERVATION

While correcting the Download section:

- Do not interfere with the existing design editor.
- Do not modify the working text insertion features.

- Do not interfere with Sync Layout functionality.
- Do not break uploaded-image insertion.
- Do not alter DOCX-to-template mapping.
- Preserve existing supported export formats.
- Do not rewrite unrelated application functionality.

Inspect the existing export and rendering architecture before making changes.

Ensure that the export process uses the actual design data and all supported elements rather than relying on an incomplete or outdated representation of the design.

7. ACCEPTANCE CRITERIA AND TESTING

The correction is successful when:

1. PNG, JPG, WebP and PDF have appropriate SVG icons.
2. The file-type dropdown displays the correct SVG icons.
3. The icons are clean, consistent and responsive.
4. PNG exports retain high image quality.
5. JPG exports retain high image quality.
6. WebP exports retain high image quality.
7. PDF exports preserve the complete design composition.
8. Tables, text, horizontal lines, shapes and uploaded images are included.
9. All supported formatting, positioning and dimensions are preserved.
10. No design elements are unexpectedly missing, clipped or distorted.
11. Exported results closely match the appearance of the designs in the editor.
12. Users can download the finished files directly to their phones or computers.
13. Downloaded files can be opened and used outside Batch Studio.
14. Existing working features remain functional after implementation.

Mandatory Export Testing

Create a test design containing:

- A formatted title with an outline.
- A subtitle and body text.
- A table containing actual text.
- A horizontal line.
- Several shapes.
- An uploaded image overlay.
- Different colours, sizes, effects and positioning.

Export the design separately as PNG, JPG, WebP and PDF.

Open and inspect every exported file.

Verify that all elements are present, correctly positioned and visually consistent with the editor.

Test on mobile as well as desktop where possible.

Do not consider the bug fixed simply because the download button works. The exported files themselves must meet the quality and completeness requirements.

8. FINAL IMPLEMENTATION REQUIREMENT

Implement the complete correction to the Download/Export section.

The objective is to make Litdom Batch Studio produce professional, high-quality downloadable designs that faithfully preserve everything the user has created.

The final result must be clean, complete, sharp, properly formatted and ready for use outside the application.

Do not sacrifice export quality or design completeness for download speed.

BUG-011 — Text Box Resizing, Scaling, Movement, and Rotation Controls

Feature/Location: Text Editing Area / Text Box Controls
PDF Visual Reference: BUG-011, Page 13

1. Correct Text Box Behaviour While Typing

Text boxes must behave intelligently and automatically accommodate the text entered by the user.

- When a user types into a text box, the box should automatically adjust to accommodate the content.
- Its dimensions should adapt as text is added or removed.
- The text should remain properly contained within the text box.
- When the user reduces the width of a text box, the text must wrap naturally onto subsequent lines rather than become unnecessarily compressed or distorted.

2. Separate Text Box Width Adjustment from Text Scaling

The text box currently has three control points along each side:

- One circular handle at the top or beginning corner.
- One circular handle at the bottom or ending corner.
- One oval-shaped handle positioned midway between the two corners.

These controls must perform different functions.

A. Middle Oval Handles — Adjust Text Box Dimensions

The middle oval handle on each side must control the dimensions of the text box.

For example, when the user drags the middle handle on the right side towards the left:

- The text box becomes narrower.
- The text automatically wraps onto additional lines to fit the reduced width.
- The text remains readable and properly contained.
- The text's font size must not automatically increase or decrease simply because the box width has changed.

The same principle applies when adjusting the other sides of the text box.

B. Four Corner Circular Handles — Scale the Text

The four corner handles must control the actual size of the text and its text box.

When the user drags a corner handle outward:

- The text must increase in size.
- Its font size must increase accordingly.
- The text box must scale appropriately with the text.
- The operation must not simply restore the text box to its previous width or undo the user's earlier width adjustment.

When the user drags a corner handle inward, the text and its box should reduce in size proportionally.

Critical correction: Resizing the text box with a middle oval handle and scaling the text with a corner handle are separate operations. Scaling must not reverse or undo a previously applied width adjustment.

3. Add a Plus (+) Movement Control

Add a clearly identifiable plus (+) control underneath the selected text box.

Its purpose is to make positioning and moving the selected text element easier and more controlled.

- The user should be able to use it to move the selected text element smoothly around the design.
- Movement must not require the user to enter text-editing mode.
- The control must not accidentally modify the text, font size, or text box dimensions.
- The text element must remain independently selectable and movable.

4. Add a Rotation Control

Add a small circular rotation control underneath the selected text box, alongside the plus (+) movement control.

The rotation control must allow the user to rotate the selected text element freely.

Required behaviour:

- Dragging the rotation control rotates the text element in the desired direction.
- The rotation must be visible in real time as the user moves the control.
- The user must be able to position text horizontally, vertically, diagonally, or at any other desired angle.
- The rotation must not distort the text or change its font size.
- The rotation must apply to the selected text element, not the entire design.

Rotation Angle Indicator

Display the current rotation angle while the user rotates the text.

Examples include:

- 0° — normal horizontal orientation.
- 45° — diagonally rotated.
- 90° — vertically rotated.
- 180° — reversed horizontal orientation.
- 270° — vertically rotated in the opposite direction.

The displayed angle must update as the user rotates the element, rather than showing a fixed value.

5. Alignment Guides During Rotation and Movement

Introduce visible alignment guides around the selected text element to help the user position and rotate it accurately.

- Show appropriate alignment guide lines while moving or rotating the element.
- Help the user identify its position relative to the design.

- Where applicable, provide visual guidance for horizontal, vertical, and centre alignment.
- Ensure the guides do not become permanent parts of the exported design.

6. Preserve Existing Text Editing Functionality

This correction must not interfere with:

- Typing and editing text.
- Automatic text-box adjustment while typing.
- Text wrapping when the box width changes.
- Font combinations.
- Font family, font size, weight, and colour settings.
- Text outlines and other supported effects.
- Text selection and positioning.
- Existing text fields and mapped content.
- Exporting the completed design.

All changes must apply only to the currently active design unless the user deliberately invokes Sync Layout.

7. Testing and Acceptance Criteria

Test the following behaviours independently:

1. Type a long sentence and confirm that the text box accommodates the content.
2. Drag a middle oval handle inward and confirm that the text wraps onto additional lines without arbitrary font-size changes.
3. Drag a corner handle outward and confirm that the text increases in size instead of returning to its previous width.
4. Repeat the operation after narrowing the text box to confirm that the previous adjustment is preserved.
5. Drag a corner handle inward and confirm proportional reduction.
6. Use the plus (+) control to move the text element smoothly.
7. Rotate the text using the circular rotation control.
8. Confirm that the angle indicator updates during rotation.
9. Confirm that alignment guides appear when appropriate and are excluded from exports.
10. Confirm that the text remains editable after resizing, scaling, moving, and rotating.
11. Confirm that other designs in the project remain unchanged.

Implementation instruction: Inspect the existing text-box architecture and its resize, scale, movement, and rotation event handlers before making changes. Correct the underlying behaviour rather than merely changing the appearance of the controls. Preserve all unrelated working functionality.

TEXT ELEMENT QUICK-ACTION TOOLBAR — FEATURE SPECIFICATION

Feature/Location: Text Editing Area / Selected Text Box

1. Overview

When the user taps a text box or selects a text element on the design surface, display a compact contextual toolbar containing four clearly identifiable SVG icons:

1. Duplicate
2. Delete
3. Nudge
4. Rotate

The toolbar should be small, neat, and positioned conveniently around or underneath the selected text box without obstructing its content.

2. Duplicate

- Display a duplicate icon.
- When tapped, create an exact copy of the selected text element.
- Preserve its text content, font, font size, colour, formatting, effects, dimensions, and other supported properties.
- Insert the duplicate as an independent, selectable element on the same design.
- The user must be able to move and edit the duplicate without changing the original.

3. Delete

- Display a delete icon using an SVG representation.
- When tapped, remove the selected text element from the current design.
- Do not delete other elements or affect other designs.
- Ensure the deletion applies to the selected element only.

4. Nudge

- Display a nudge/movement icon.
- When tapped, activate a movement mode for the selected text element.
- Allow the user to move the text freely around the design using finger movement.
- Movement should be smooth and responsive.

- The operation must not accidentally change the text, font size, formatting, or dimensions.

5. Rotate

- Display a rotation icon.
- When tapped, activate rotation mode for the selected text element.
- Allow the user to rotate the text using a circular finger gesture.
- Rotation should respond continuously to the direction and movement of the user's finger.
- Display the current rotation angle while rotating, including values such as 45°, 90°, 180°, and 270°.
- Rotation must not distort the text or change its font size.

6. Interface and Behaviour

- Show the four icons when a text element is selected.
- Keep the toolbar compact and visually consistent.
- Use proper SVG icons, not emojis.
- Make the icons distinguishable and easy to tap on mobile screens.
- Hide the contextual toolbar when the text element is deselected.
- Ensure the toolbar does not obstruct normal text selection, typing, or editing.

7. Integration and Non-Regression

Integrate these actions with the existing text-box controls described in BUG-011.

Avoid creating duplicate or conflicting movement and rotation mechanisms. The Nudge and Rotate toolbar actions should work with the existing movement, rotation, and angle-indicator behaviour.

All actions must affect only the selected element on the currently active design. Other designs must remain unchanged unless the user deliberately invokes Sync Layout.

Preserve all existing text editing, resizing, scaling, font formatting, and export functionality.

8. Testing and Acceptance Criteria

1. Select a text box and confirm all four icons appear.
2. Duplicate it and confirm the copy retains the original formatting and remains independently editable.
3. Delete the duplicate and confirm the original remains.
4. Activate Nudge and move the text to different positions.
5. Activate Rotate and rotate the text using a circular finger gesture.

6. Confirm the angle indicator updates during rotation.
7. Confirm the toolbar disappears when the element is deselected.
8. Confirm all four actions work correctly on mobile and desktop.
9. Confirm unrelated elements and other designs remain unchanged.

BUG-012 — Text Editing Modes, Formatting Controls, Font Library, and Direct Canvas Editing

Feature/Location: Text for This Design / Text Editing Area
PDF Visual Reference: BUG-012, Page 14

1. Overview

The text editing system must support two independent but interconnected editing modes.

- **Mode A:** Whole-text-box editing. Changes apply to the entire text element.
- **Mode B:** In-text editing. Changes apply to text highlighted by the user's native keyboard selection.

Both modes must work independently, interchangeably, and continuously. Using one mode must never disable the other.

The text must be edited directly on the design canvas, similar to Canva and Adobe editing workflows. Do not force the user to edit through a separate text field or spreadsheet cell.

2. Mode A — Whole-Text-Box Editing

When the user taps a text element once, display the main editing toolbar.

Mode A contains:

1. Edit
2. Style
3. Font
4. Size
5. Colour
6. Effects
7. Format

A. Edit

The Edit button enters Mode B, allowing the user to edit the actual text content and format selected words or passages.

B. Style

Retain the existing working text-style functionality, including the available heading and paragraph styles.

Do not unnecessarily modify the existing working style assignments.

C. Font

The Font control must open a list of available fonts.

- Allow selection of fonts such as Cinzel and Arial.
- Selecting a font changes the entire text element.
- Include an Upload Font option.
- Do not force users to upload a font before they can select an existing font.

D. Size

Allow the user to increase or decrease the font size of the entire text element.

E. Colour

Provide the existing colour controls and add fully functional gradient colour support.

- Allow solid-colour selection.
- Allow gradient selection.
- Allow the user to configure the gradient using the supported gradient controls.
- Apply the selected colour or gradient to the entire text element.

Mode A must not be missing gradient colour functionality.

F. Effects

Correct the existing Mode A effects functionality.

Effects must apply to the entire text element, including all words in the text box.

Requirements:

- Effects must render smoothly and cleanly.

- Outlines must have smooth, consistent edges, without jagged, crystalline, fragmented, or distorted appearances.
- All effect controls must work, including intensity, blur, and opacity.
- Changes to each control must produce the corresponding visible change.
- Effects must update properly when changed.
- Do not require the user to enter Mode B to apply an effect to the entire text element.

G. Format

The Format control must support:

Text styling

- Bold.
- Italic.
- Underline.
- Other existing supported text-formatting controls.

Text alignment

- Align left.
- Align centre.
- Align right.

Lists

- Bulleted list.
- Numbered list.
- Remove bullet or numbered-list formatting where applicable.

Spacing

- Increase and decrease line spacing.
- Increase and decrease letter spacing.

The list controls must work on the entire text element when no individual text selection is being edited.

Line spacing controls must adjust the distance between lines of text. Letter spacing controls must adjust the distance between individual characters.

Do not confuse these two spacing properties.

3. Mode B — In-Text Editing

Mode B appears when the user selects a text element and taps Edit in Mode A.

Its purpose is to allow direct editing of the text content and formatting of selected words or passages.

A. Direct Canvas Text Editing

The user must be able to:

- Tap Edit.
- Edit the actual text directly inside the selected text box on the design surface.
- Use the phone's native keyboard.
- Delete existing text using Backspace.
- Add new text.
- Replace words and sentences.
- Correct spelling and clean up existing content.
- Continue typing without opening a separate text-entry field.

The text box must update as the user types.

Its dimensions must automatically accommodate the changing content, following the text-box resizing requirements already documented in BUG-011.

Do not redirect editing to the Upload Sheet, spreadsheet cell, or another detached text field.

B. Native Keyboard Text Selection

Use the phone's native keyboard and text-selection handles.

- The user must be able to highlight individual words, multiple words, or passages.
- Do not require the user to tap words individually.
- Do not require a separate selection-range operation.
- Do not introduce an artificial selection system that conflicts with the phone's keyboard.

Remove the following existing interface elements:

- Select All.
- Select Range.
- Multi-Word.
- The associated selection controls and obstructive black overlay.
- The separate All Words 12 tag or word-count tag.

Retain the Done button so the user can finish editing.

C. Mode B Formatting Toolbar

When Mode B is active, display its additional formatting controls in one horizontally scrollable row directly above the phone's keyboard.

Do not open another large dashboard or obstructive panel.

The controls must include:

- Bold.
- Italic.
- Underline.
- Font selection.
- Font size increase and decrease.
- Uppercase/lowercase/mixed-case conversion.
- Effects for highlighted text.

Retain the existing supported additional text controls where appropriate.

When the user highlights text using the phone's native selection handles and taps a formatting control, apply the change to the highlighted portion only.

Examples:

- Highlight one word and tap Bold: only that word becomes bold.
- Highlight a word and tap Italic: only that word becomes italic.
- Highlight a passage and change its font: only the highlighted passage changes.
- Highlight a word and change its colour or effect: only that word changes.

If the selected text already has the requested formatting, allow the user to toggle that formatting off.

D. Uppercase, Lowercase, and Mixed Case

Provide a case-conversion control that opens a compact popup containing three distinct options:

1. **Lowercase:** Convert all selected text to lowercase.
2. **Uppercase:** Convert all selected text to uppercase.
3. **Mixed:** Capitalise the first letter of every word.

These options must work on the text highlighted using the phone's native keyboard selection.

Do not omit Mixed Case.

E. Mode B Effects

Mode B must have its own functional text-effect capability.

- Apply effects only to the highlighted text.
- Render effects smoothly.
- Make intensity, blur, and opacity functional.
- Do not apply the effect to unselected words in the text element.

Mode B effects must not replace or disable Mode A effects.

4. Shared Font Library for Both Modes

Mode A and Mode B must use one shared font library.

Mode A

- Selecting a font changes the entire text element.
- Users can select an existing font or upload a new one.

Mode B

- Selecting a font changes only the highlighted text.
- Users can select an existing font or upload a new one.

Shared-library requirement:

A font uploaded through either mode must immediately become available to both modes.

For example:

- Upload a font through Mode A.
- Find and use that font through Mode B.
- Upload another font through Mode B.
- Find and use that font through Mode A.

Do not maintain separate font libraries for the two modes.

Uploaded fonts must be retained and remain available across future designs and sessions, subject to the application's existing storage architecture.

Validate uploaded font files and provide clear feedback for unsupported or invalid files. Do not silently substitute unrelated fonts.

5. Mode A and Mode B Must Remain Independent

This is a critical requirement.

Mode A and Mode B must work before and after one another.

Examples:

Scenario 1

1. Select a text box.
2. Use Mode A to change the font to Cinzel.
3. Enter Mode B.
4. Highlight one word and make it bold.
5. Tap Done.
6. Return to Mode A and change the entire text box's colour.

All operations must work.

Scenario 2

1. Select a text box.
2. Enter Mode B.
3. Change the font of one highlighted word.
4. Tap Done.
5. Select the same text box again.
6. Use Mode A to change the entire text element's font size, effects, colour, or formatting.

All Mode A controls must remain functional.

Scenario 3

1. Use Mode A to apply an effect to the entire text element.
2. Enter Mode B.
3. Apply a different effect to a highlighted word.
4. Finish editing.
5. Return to Mode A.

The overall text formatting and the individual word formatting must remain properly represented.

Mode B must never permanently lock the text element into individual-word editing.

Mode A must never require the user to re-enter Mode B simply to change the entire text box.

6. Text Formatting Scope

Maintain a strict distinction between the two modes:

Operation	Mode A	Mode B

Font family	Entire text element	Highlighted text
Font size	Entire text element	Highlighted text
Solid colour	Entire text element	Highlighted text
Gradient colour	Entire text element	Highlighted text, where supported
Effects	Entire text element	Highlighted text
Bold	Entire text element	Highlighted text
Italic	Entire text element	Highlighted text
Underline	Entire text element	Highlighted text
Alignment	Entire text element	Selected paragraphs, where supported
Bullet lists	Entire text element or applicable paragraphs	Selected paragraphs
Numbered lists	Entire text element or applicable paragraphs	Selected paragraphs

Line spacing	Entire text element	Selected paragraphs, where supported
Letter spacing	Entire text element	Highlighted text
Case conversion	Not required	Highlighted text
Direct text replacement	Through Edit to enter Mode B	Directly on canvas

A formatting operation must respect its intended scope. Applying formatting to highlighted text must not unintentionally alter the rest of the text element.

7. Style and Layouts Area

The existing Style and Layouts area does not appear to be functioning correctly.

For this correction:

- Do not attempt to rebuild or redesign it.
- Do not introduce new functionality into it.
- Leave the area untouched for a later development session.
- Do not allow it to interfere with the Text for This Design functionality.

Do not delete the Style and Layouts area at this stage.

8. Non-Regression Requirements

Preserve the following:

- Existing working text styles.
- Add Heading, Add Subheading, and Add Body Text.
- Font combinations.
- Shared font library.
- Text-box automatic adjustment and wrapping.
- Text-box resizing and scaling.
- Text movement and rotation.

- Duplicate and delete controls.
- Sync Layout and Sync Text Field functionality.
- DOCX mapping and its existing working behaviour.
- Existing projects and templates.
- Supported export formats.

All ordinary formatting changes must affect only the currently active design. Changes to other designs must occur only when the user deliberately invokes Sync Layout.

9. Implementation and Testing

Inspect the existing text editor architecture before modifying it.

Pay particular attention to:

- The relationship between Mode A and Mode B.
- How the text element stores whole-element formatting.
- How individual character and word formatting is stored.
- How native keyboard selections are detected and applied.
- How text content updates on the design canvas.
- How font libraries and uploaded fonts are managed.
- How effect controls communicate with the rendering engine.

Do not implement Mode A and Mode B as competing systems or allow one to overwrite the other's state.

Test the following:

1. Whole-text-box font changes.
2. Individual-word font changes.
3. Shared font uploads from both modes.
4. Whole-text-box colour and gradient.
5. Individual-word colour and gradient, where supported.
6. Whole-text-box effects.
7. Individual-word effects.
8. Intensity, blur, and opacity independently.
9. Smooth effect rendering.
10. Whole-text-box bold, italic, and underline.
11. Individual-word bold, italic, and underline.
12. Bulleted lists.
13. Numbered lists.
14. Removing list formatting.
15. Line spacing adjustments.
16. Letter spacing adjustments.
17. Uppercase, lowercase, and mixed-case conversion.

18. Direct typing and deletion on the design canvas.
19. Native keyboard selection and formatting.
20. Returning from Mode B to Mode A and confirming all Mode A controls still work.
21. Repeatedly switching between both modes without losing formatting or text.
22. Confirming that the text box updates as the user types.
23. Confirming that the Done button finishes editing correctly.

Finally, inspect exported designs to ensure that the text content, formatting, gradients, effects, spacing, and individual-word styling are preserved.

Final implementation rule: Do not merely make the controls visible. Every control must perform its intended operation, and both editing modes must remain fully functional throughout the editing process.

10. Mode A Format Popup — Precision Nudge Controls

Enhancement-03

Visual Reference pdf: page 14

Add a feature called **Nudge** at the bottom of the Format popup in Mode A.

Interface

Display four directional arrow buttons:

- ↑ Move upwards.
- ↓ Move downwards.
- ← Move left.
- → Move right.

Use clean, consistent SVG icons.

Functionality

Each arrow must move the entire selected text element in small, precise increments in the indicated direction.

- Each tap moves the element a small distance.
- Repeated taps produce repeated small movements.
- The movement must be slow and controlled enough to allow precise positioning.
- The four directions must work independently.

- Movement must preserve the text content, font, size, colour, effects, formatting, and dimensions.

The purpose is to help the user position an element accurately when ordinary dragging produces movements that are too large.

Scope

- Mode A Nudge moves the entire selected text element.
- It must not change individual words or their formatting.
- It must not move other elements or affect other designs.
- It must work alongside ordinary dragging and the existing movement controls.

Testing

1. Tap a text box and open Mode A Format.
2. Locate the four Nudge arrows at the bottom.
3. Tap each arrow and confirm movement in the correct direction.
4. Tap an arrow repeatedly and confirm consistent small increments.
5. Confirm that the text element can be positioned more precisely than with ordinary dragging.
6. Confirm that its formatting and dimensions remain unchanged.

BUG-013 — Application Freezing, Persistent Studio Loading, and Strict Non-Regression Protection

Feature/Location: Application Performance / Deployment / Studio Loading
PDF Visual Reference: BUG-013, Page 15

1. Deployment Delays and Temporary Freezing

The application sometimes appears to freeze or behave as though newly deployed features are not working.

Observed behaviour:

- After deploying changes, some previously existing functionality continues working.
- Newly implemented functionality may initially appear unresponsive or unavailable.

- After some time, the new functionality begins working correctly without another code change.
- This creates uncertainty about whether the deployment succeeded or whether a feature is actually broken.

Required correction

Inspect the deployment and application-loading architecture.

- Ensure the application loads the latest successfully deployed version consistently.
- Investigate stale cached files, delayed asset updates, service-worker behaviour, and deployment propagation where applicable.
- Prevent users from unknowingly running a mixture of old and new application resources.
- If an update is still loading, provide an appropriate loading or update status rather than making the application appear broken.
- Ensure the application becomes usable once the required resources are ready.
- Do not automatically reload in the middle of an active editing session if doing so could cause loss of unsaved work.

Do not assume that every temporary deployment delay is a code defect. Identify the actual cause before changing the deployment mechanism.

2. Studio Remains on Loading Studio After Saving Large Documents

Observed behaviour:

After saving or downloading a large document, returning to the application can cause the Studio loading screen to remain visible for an unusually long time.

Sometimes the user has to:

- Refresh the application, or
- Leave the application and reopen its link,

before the Studio loads normally again.

Required correction

Investigate why saving or processing large documents leaves the Studio in a persistent loading state.

Inspect:

- Large-file processing.
- Memory consumption and resource cleanup.
- Save and download completion handling.
- Loading-state management.
- Background processing and pending asynchronous operations.
- Restoration of the editor after saving or downloading.
- Handling of large images, documents, and project data.

The application must:

- Return to the usable Studio interface after a successful save or download.
- Clear completed loading states correctly.
- Avoid remaining indefinitely on Loading Studio.
- Handle large documents without unnecessarily blocking the entire application.
- Display a meaningful error or recovery option if an operation genuinely fails.
- Preserve saved work and prevent duplicate saves or accidental data loss.

The user should not have to refresh the application or reopen its link every time a large document is saved or downloaded.

3. STRICT NON-REGRESSION PROTECTION

This is a mandatory instruction.

All other existing application functionalities are currently working well and must be preserved.

AI Studio must not use this correction as an opportunity to redesign, rewrite, optimise, replace, or restructure unrelated parts of the application.

- Do not modify unrelated working features.
- Do not replace working components unnecessarily.
- Do not change existing project structures or data formats without a demonstrated technical necessity.
- Do not alter working DOCX mapping, project management, templates, uploads, exports, navigation, or other established functionality while addressing these issues.
- Inspect dependencies before changing shared application components.
- Limit changes to the identified deployment, loading, and resource-management problems.

If a shared component must be changed to correct one of these issues, inspect its dependencies first and regression-test the existing features that depend on it.

4. Priority: Text Effects Must Work Correctly

The text-effect corrections already documented in BUG-012 remain a major implementation priority.

AI Studio must pay particular attention to making the effects work correctly and render professionally.

Verify that:

- Effects render smoothly without rough, jagged, crystalline, or fragmented outlines.
- Intensity works.
- Blur works.
- Opacity works.
- Mode A effects apply to the entire text element.
- Mode B effects apply only to highlighted text.
- Effect changes update visibly and correctly.
- The effect controls do not merely appear functional while failing to change the result.

Do not sacrifice effect quality or correctness for superficial interface improvements.

5. Final Testing and Acceptance Criteria

Before considering these corrections complete:

1. Deploy an updated application and verify that the latest version loads consistently.
2. Test newly deployed features immediately after deployment.
3. Confirm that stale application resources do not prevent updates from taking effect.
4. Save and download a large document.
5. Return to the Studio and confirm that it loads without requiring a manual refresh.
6. Repeat the test with different large documents and projects.
7. Confirm that saved work is preserved.
8. Test the Mode A and Mode B text effects, including intensity, blur, and opacity.
9. Confirm that existing working application functionalities remain unaffected.
10. Regression-test any shared components that were necessarily modified.

Final instruction: Fix the identified technical problems without introducing new problems into the existing application. Preserve working functionality and concentrate implementation effort on reliable deployment, correct Studio loading, and fully functional, professionally rendered text effects.

Robust State Management for Dual-Mode Text Editing: Ensure that transitioning between Mode A (whole-box editing) and Mode B (in-text/keyboard selection) handles overlapping styles cleanly so that individual word formatting (such as bold or font overrides) is never inadvertently wiped out or locked when applying whole-box changes.

- Comprehensive Export Fidelity Testing: Because the specifications emphasize preserving every element (tables, overlays, horizontal lines, shapes, and custom SVG icons), establish an automated or rigorous manual checklist comparing the canvas rendering directly against exported PNG, JPG, WebP, and PDF files to catch clipping or missing layer issues early.

- Precise Asynchronous Handling for Large Files: To resolve BUG-013 (persistent studio loading after saving large documents), implement explicit timeout recovery, memory cleanup, and status indicators for background serialization so the UI never appears indefinitely frozen during heavy file operations.

- Independent Sync Scopes: Verify that Sync This Design’s Layout and Sync Text Fields Layout strictly maintain separate action flows, thumbnail selections, and default "This Page" constraints to prevent accidental batch overwrites across projects.